

2.3. Creación de scripts

Beatriz Rey Solaz
Alejandro Rodríguez Ortega
Departamento de Ingeniería Gráfica

Índice

0- Previo al tema: visualización de controladores en RV.
1- Creación de scripts.
2- Métodos de inicialización.
3- Métodos de actualización.
4- Primeros pasos: debug, consola.
5- Variables.
6- Modificaciones en las transformaciones.
7- Eventos asociados a detección de colisiones.
8- Acceso a componentes del propio GameObject.
9- Acceso a componentes de otros GameObjects.
10 - Uso de corrutinas.
11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.
12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

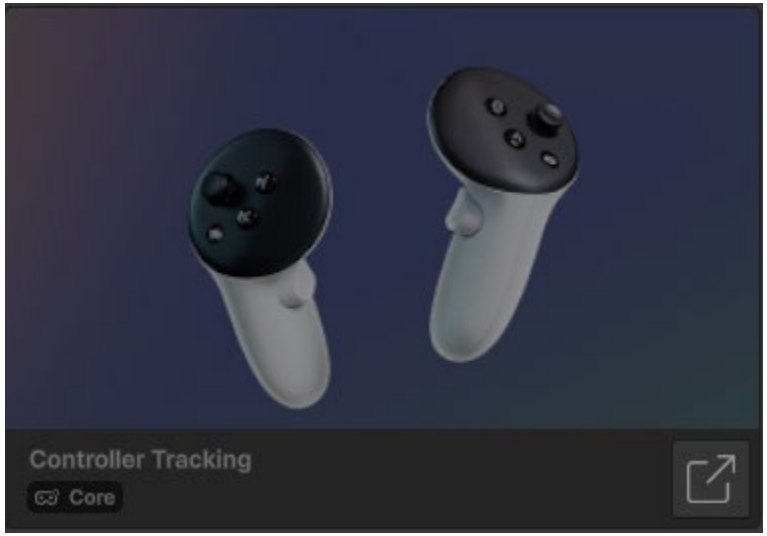
11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Añadir building block Controller Tracking

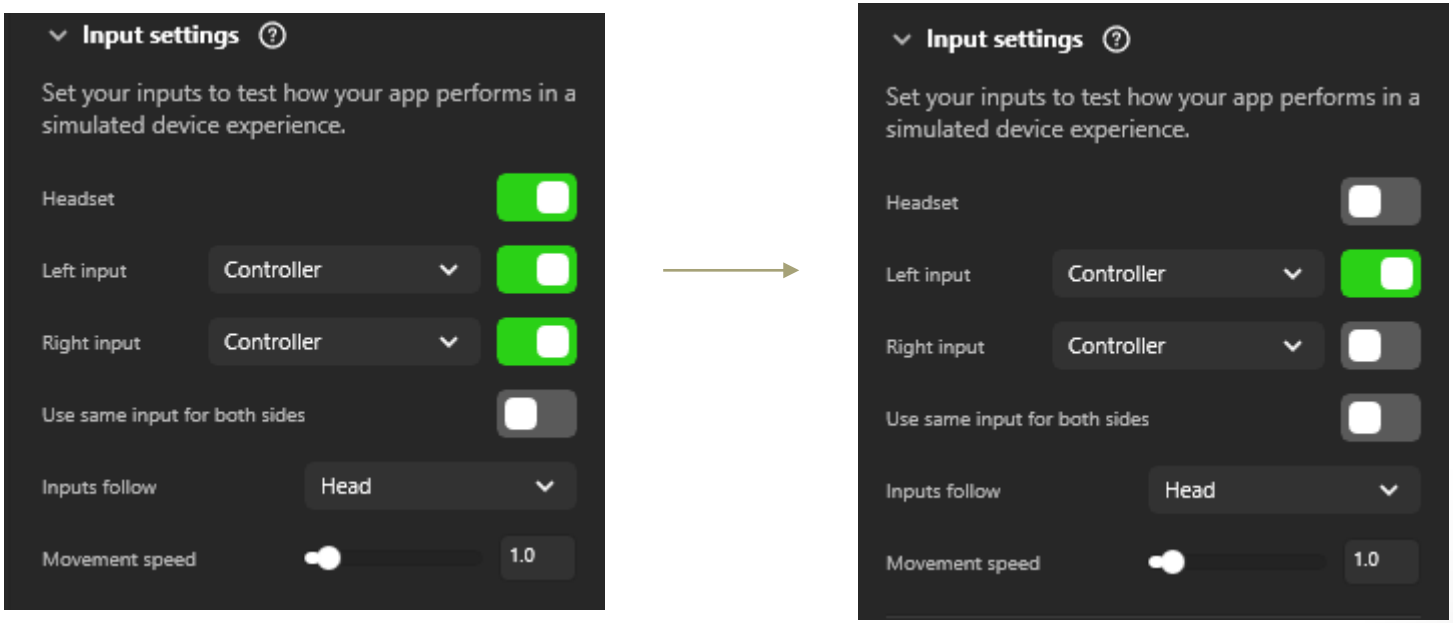
- Instancia el modelo 3D del controlador detectado y lo hace visible en la escena.
- Comentaremos con más detalle lo que aporta este building block en el tema 3.2.



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Opciones del simulador para controladores

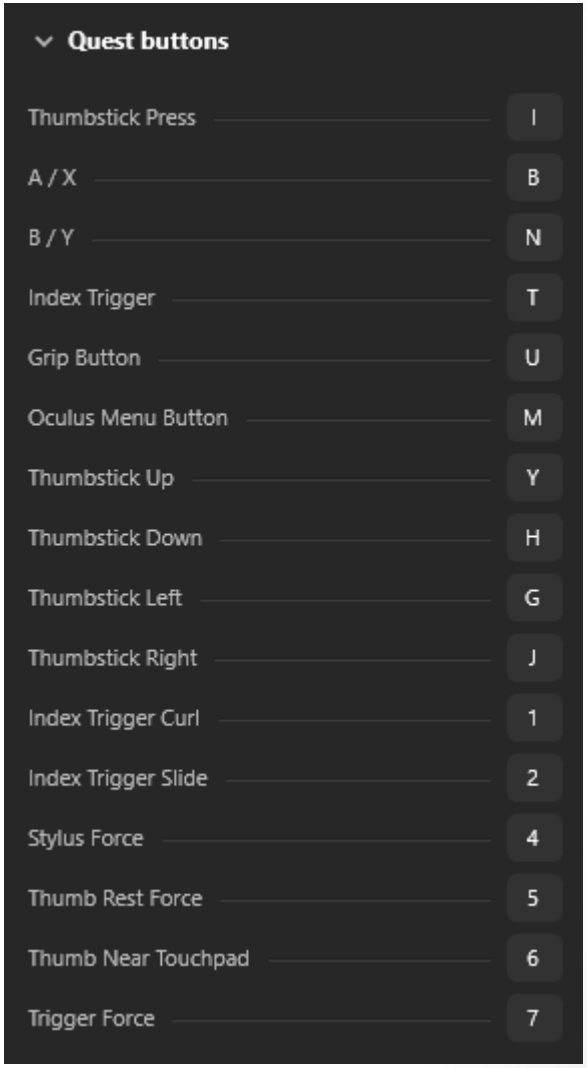
- Puedo manejar únicamente un controlador y modificar su transformación con las mismas teclas que la cámara (WSAD y movimiento del ratón con botón derecho pulsado)



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Opciones del simulador para controladores

- Puedo simular pulsaciones de botones del controlador



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Introducción a C# en Unity

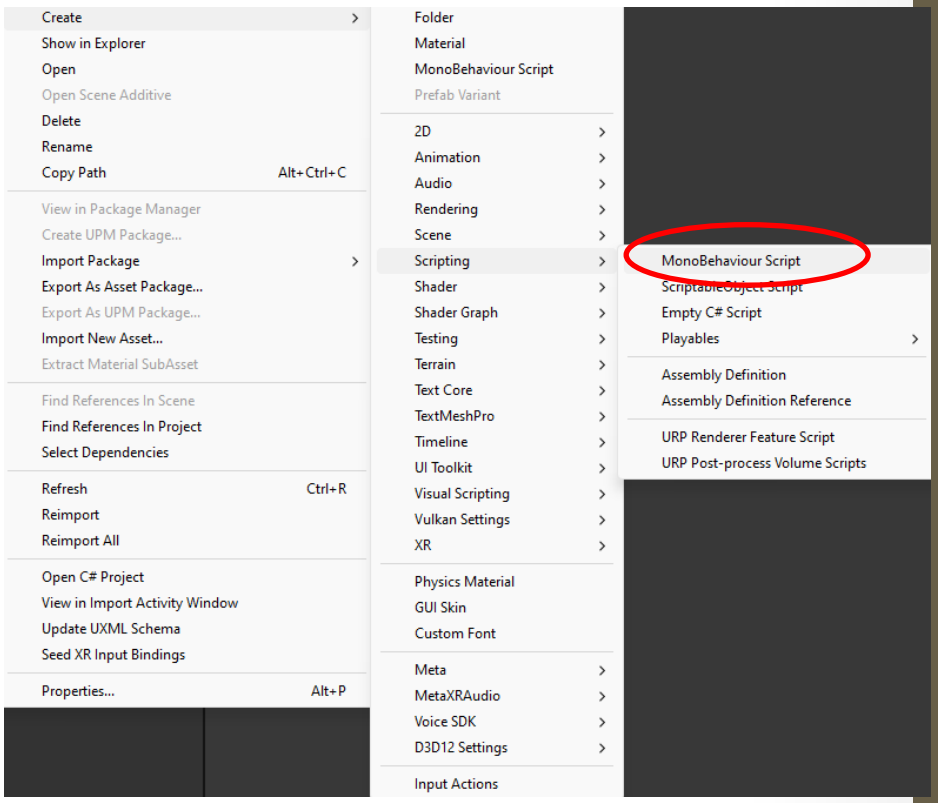
- **C#** es un lenguaje creado en el año 2000 con la plataforma .NET (Microsoft).
- Utiliza características de C++, Java y Visual Basic.
- Orientado a objetos.

- Unity permite utilizar las características más importantes de C#.
- Además, proporciona un **conjunto amplio de clases y posibilidades** que podemos utilizar en nuestras aplicaciones.
- En Windows, se proporciona el entorno de trabajo Visual Studio.

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Crear un script

- Crear carpeta “Scripts” en Assets.
- RMB – Create – Scripting – MonoBehaviour Script.
- MUY IMPORTANTE:
  - En Unity los scripts han de ir asociados a un GameObject → **Arrastrar el script sobre el GameObject al que deba ir asociado**.
  - Recomendable que el nombre del fichero coincida con el nombre de la clase contenida en el mismo.



Estructura de un fichero C#

- **MonoBehaviour Class**: para definir scripts asociados a GameObjects o Prefabs.

test.cs

Archivos varios

test

```
1   using UnityEngine;
2
3   public class test : MonoBehaviour
4   {
5       // Start is called once before the first execution of Update after the MonoBehaviour is created
6       void Start()
7       {
8
9       }
10
11      // Update is called once per frame
12      void Update()
13      {
14
15      }
16  }
17
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Desarrollo de scripts en C#

- Los scripts de Unity derivan de la clase MonoBehaviour.
- **Los scripts son componentes de los GameObjects.**
  - "this" se refiere al script, no al GameObject.
- Un mismo GameObject puede tener varios scripts.
- "**Event functions**": métodos que Unity activa en respuesta a eventos que ocurren durante la ejecución de la aplicación.
- Métodos más importantes:
  - Awake(), Start()
  - Update(), FixedUpdate(), LateUpdate()
  - OnCollisionEnter(), OnCollisionStay(), OnCollisionExit()
  - OnTriggerEnter(), OnTriggerStay(), OnTriggerExit()
  - OnBecameVisible(), OnBecameInvisible()
  - OnEnable(), OnDisable()

https://docs.unity3d.com/Manual/ExecutionOrder.html

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Métodos de inicialización

- Void Awake() {…}
  - Es el primer método que **se ejecuta cuando se inicializa el objeto.**
  - Debe usarse como sustituto del constructor (para inicializar las variables).
  - Se ejecuta **antes de cualquier Start()** de todos los objetos.
    - Útil para inicializar referencias cruzadas entre objetos.

- Void Start() {…}
  - Es un **segundo paso de inicialización** (después de todos los Awake() y antes que cualquier Update()).
  - **Utilizando Awake() y Start(), los objetos se pueden inicializar en dos pasos.**
  - Start() sólo se ejecuta si el GameObject está activo en el Inspector (tick marcado).

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Actualización de GameObjects

- Una de las principales misiones de un script es **refrescar o actualizar de la forma adecuada su GameObject asociado.**

- Esta actualización puede consistir en un cambio de posición, escala, rotación, fuerzas, colores, estados…

- Hay distintos métodos disponibles:
  - Update()
  - FixedUpdate()
  - LateUpdate()

Métodos de actualización

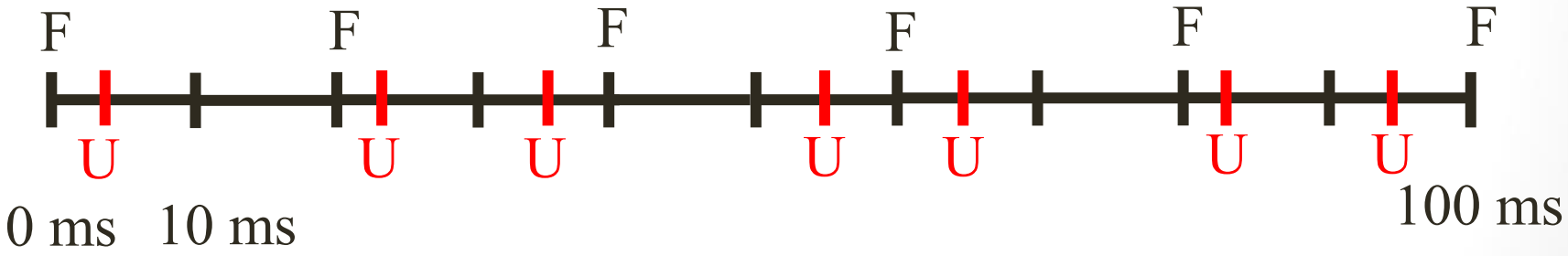
- Void Update() {…}
  - Rutina más común para actualizaciones.
  - Se llama **una vez por frame**.
  - La eficiencia del código incluido en este método afectará de manera importante los frames por segundo (FPS).
  - **Perfecto para actualizar objetos no físicos y recibir interacciones entre usuarios.**
  - Sólo se ejecutará si el objeto está activo (enabled).

Métodos de actualización

- Void FixedUpdate() {…}
  - Se llama a **intervalos regulares** (por defecto, cada 20 ms)**.**
  - **Las actualizaciones de físicas se realizarán después FixedUpdate.**
    - → Es el lugar adecuado para incluir código relacionado con cambios en el Rigibody del GameObject (p. ej., para aplicar fuerzas).
  - Se puede llamar a FixedUpdate con más o menos frecuencia que a Update.
    - Update se ejecuta de forma variable.
    - FixedUpdate se llama de forma constante.

Métodos de actualización

- Ejemplo de posible distribución de updates y fixed updates en 100 ms.
  - F: Fixed update
  - U: Update



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Métodos de actualización

- Void LateUpdate() {…}
  - **Se llama cuando todos los Update de todos los objetos han sido ejecutados.**
  - Lugar oportuno para actualizar las posiciones de los objetos después de cualquier cambio que haya sido realizado en Update().
  - **Se ejecuta cada frame.**
  - Ejemplo de uso: cuando un objeto sigue a otro en la escena, así sabemos que el primer objeto se ha movido en un Update. El segundo objeto solo actualiza su posición en LateUpdate(), cuando el primero ya se ha movido.

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

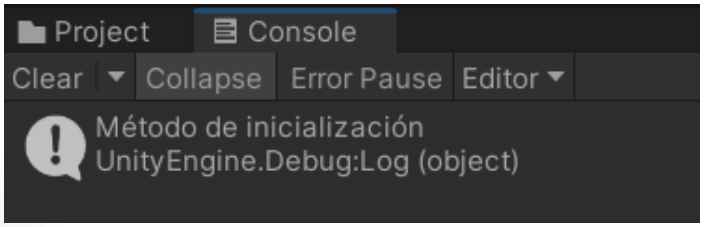
12- Scripts para personalizar aspectos relacionados con las Meta SDK.

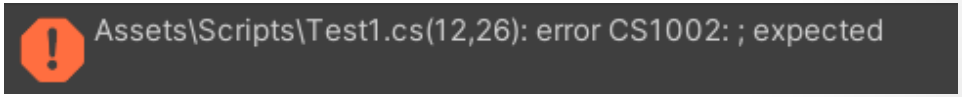
Debug

- Es posible mostrar mensajes desde nuestro código para mostrar información que ayude a depurar el funcionamiento de nuestros scripts.

```
void Start () {
        Debug.Log("Método de inicialización");
}
```

- El texto introducido en Debug.Log se muestra en la consola.
- En la consola también se muestran mensajes para informar de errores que detecta Unity.



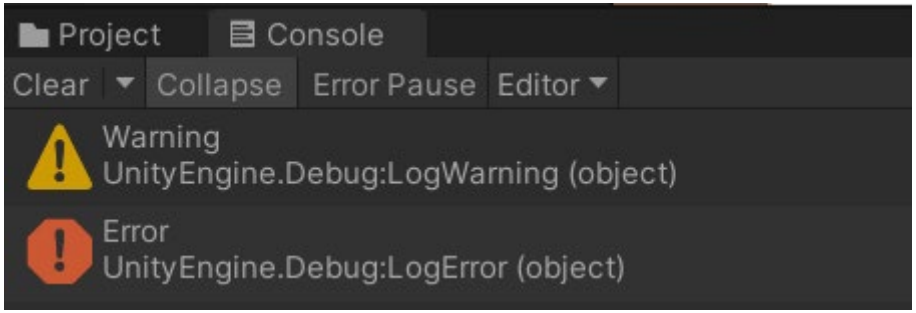


Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Warnings - Errors

- Igualmente, nosotros podemos mostrar mensajes como Warnings o Errors desde nuestro código.

```
void Start () {
        Debug.LogWarning("Warning");
        Debug.LogError("Error");
}
```



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Departamento de Ingeniería Gráfica

Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Definición de variables

```
public class Test : MonoBehaviour {
        public string myName;

        void Start() {
                Debug.Log("Mi nombre es "+ myName);
        }
}
```

- El código para definir las variables se encuentra dentro de la clase, antes de los métodos.
- Tipos de variables: string, int, float, Vector2, Vector3…

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Inicialización de variables

```
public class Test : MonoBehaviour {
        public string myName;

        void Start() {
                myName = "Bea";
                Debug.Log("Mi nombre es: "+myName);
        }
}
```

- Las variables se pueden inicializar por código en Start

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Ejemplo de inicialización de integer

```
public class Test : MonoBehaviour {
        public int counter;

        void Start() {
                counter=0;
                Debug.Log("Contador: "+counter);
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Ejemplo de inicialización de Vector3

```
public class Test : MonoBehaviour {
        public Vector3 posicion;

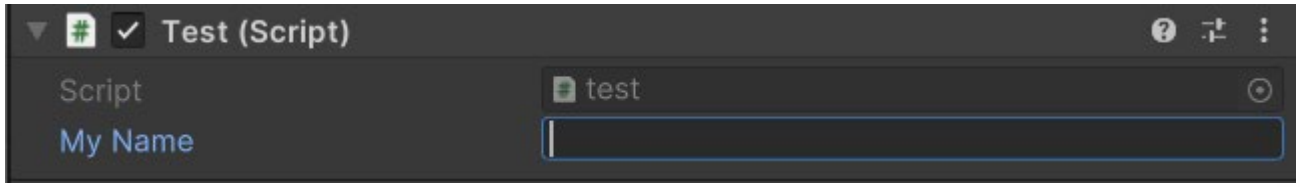
        void Start() {
                posicion = new Vector3(1.0f, 2.0f, 0.0f);
                 Debug.Log("Posición: "+posicion);
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Variables públicas y privadas

- Variables públicas: es posible acceder a las mismas desde otros scripts.
- Variables privadas: solo es posible acceder a estas variables desde el script donde se crearon.

- Las variables públicas son visibles en el Inspector, y es posible asignarles allí su valor (sin necesidad de utilizar código para ello).

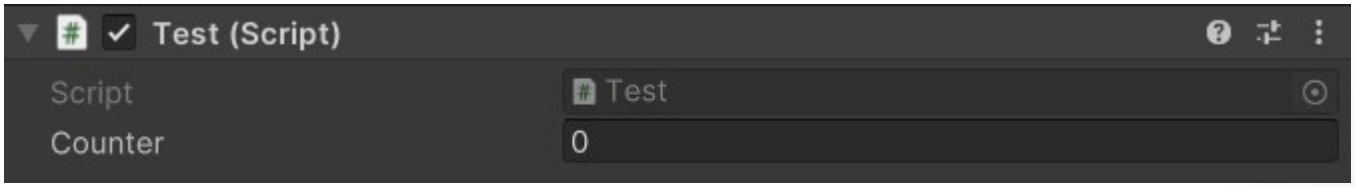


Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Control de apariencia de variables en el Inspector

- Es recomendable no poner una variable como pública si no se va a utilizar desde otros scripts.
- Si queremos que se vea desde el Inspector para asignarle un valor, pero no es pública, se puede añadir el atributo[SerializeField]

```
public class Test : MonoBehaviour {
        [SerializeField]
        private int counter;
}
```



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

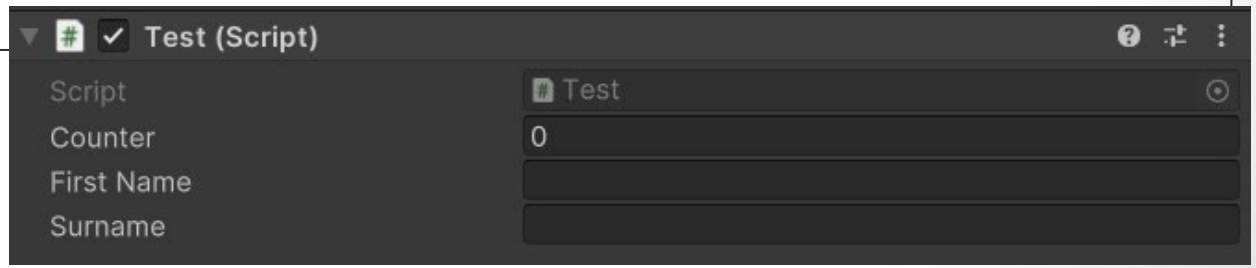
Control de apariencia de variables en el Inspector

- El atributo [SerializeField] se ha de escribir delante de cada una de las variables a las que lo queramos aplicar.

```
public class Test : MonoBehaviour {
        [SerializeField]
        private int counter;

        [SerializeField]
        private string firstName;

        [SerializeField]
        private string surname;
}
```



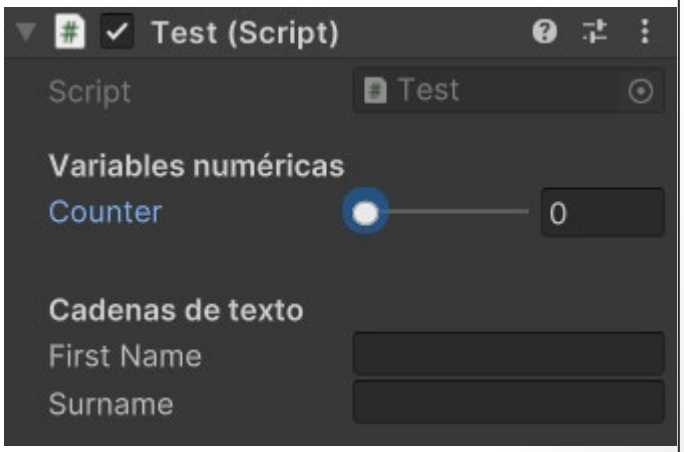
Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Control de apariencia de variables en el Inspector

- [Space] → Para dejar espacios extra
- [Header] → Para añadir cabecera en grupo de variables
- [Range] → Para delimitar el rango de posibles valores

```
public class Test : MonoBehavior {
        [Header("Variables numéricas")]
        [SerializeField]
        [Range(0,100)]
        private int counter;

        [Space]
        [Header("Cadenas de texto")]
        [SerializeField]
        private string firstName;
        [SerializeField]
        private string surname;
}
```

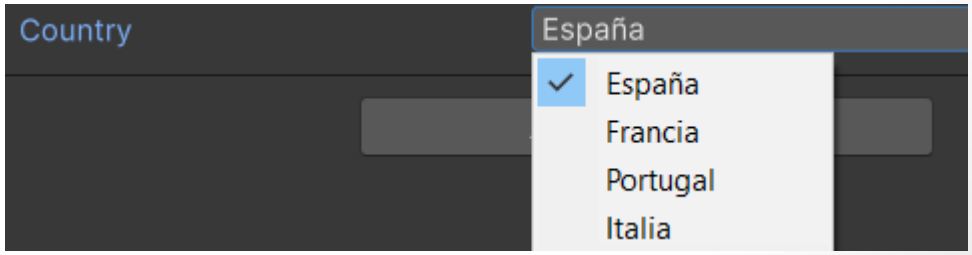


Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Tipos enumerados

- Cuando una variable puede tomar un valor entre distintas opciones, se crea un tipo enumerado.

```
public class Test : MonoBehavior {
        [SerializeField]
         private CountryOptions country;
          public enum CountryOptions
          {
             España,
             Francia,
             Portugal,
             Italia
          }
}
```



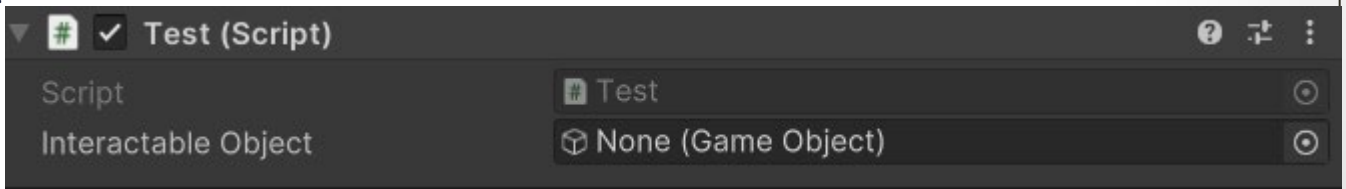
Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Variables GameObject

- Es posible crear variables de tipo GameObject para acceder a variables y métodos públicos de otros GameObjects.
- Importante: es necesario inicializar estas variables desde el editor o por código para poderlas utilizar.
  - Desde el editor, se arrastra el GameObject desde la escena o desde la jerarquía para asignarlo a la variable.
- Si no se han inicializado, la variable toma el valor “null”.

```
public class Test : MonoBehavior {
        public GameObject interactableObject;

        void Start() {
                if (interactableObject == null) {
                    Debug.LogWarning("interactableObject no inicializado");
                }
        }
}
```



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

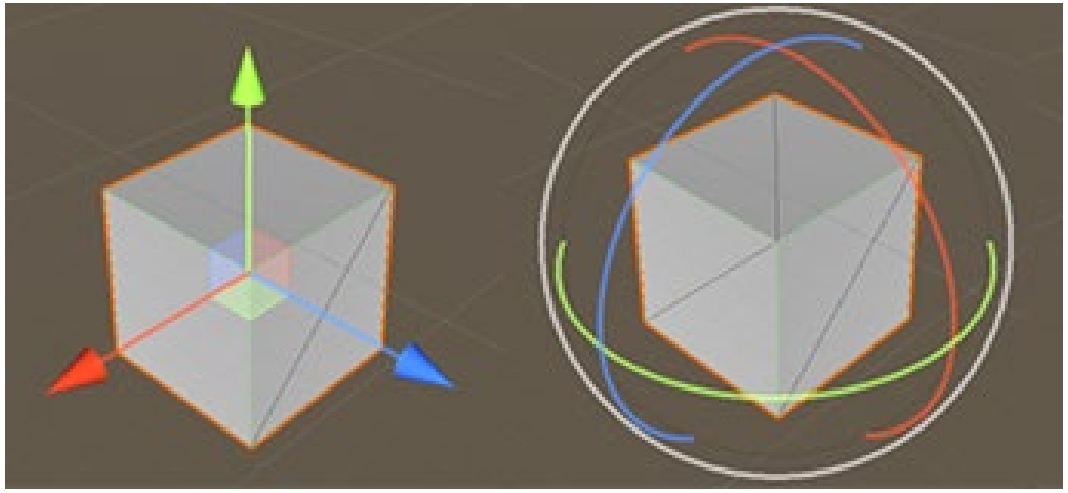
10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Formas de controlar movimiento de GameObjects

- En un entorno, los objetos pueden moverse o rotar:
  - Controlados por scripts (en este caso, es el código de la función Update el que determina cómo se mueven).
  - Controlados por la física (fuerza de gravedad, otras fuerzas, colisiones…)
  - Controlados por una animación.

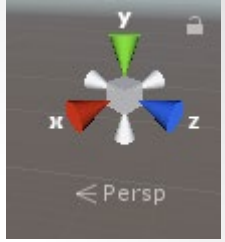


Modificaciones en la transformación

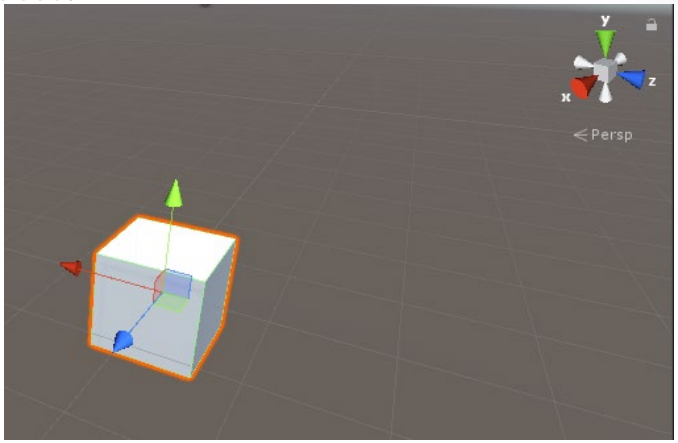
- La componente Transform incluye posición, rotación y escala de un objeto.
- Cada objeto de la escena tiene una transformación que se puede manipular.
- Cada GameObject con su transformación puede tener un padre, también con su transformación. Esto permite aplicar posición, rotación y escala de forma jerárquica.
- Es posible controlar la transformación de objetos desde el código y sin aplicar física. Métodos principales:
  - Transform.Translate
  - Transform.Rotate

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Transform.Translate

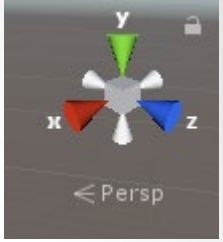


- Método de la componente Transform.
- public void Translate(Vector3 translation, Space relativeTo = Space.Self);
- Cambia la transformación en la dirección y distancia indicadas en “translation”.
  - “relativeTo” vacío o Space.Self: movimiento en los ejes locales.
  - “relativeTo” = Space.World: desplazamiento en el sistema de coordenadas del mundo.



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Transform.Translate




```
using UnityEngine;
using System.Collections;

public class ExampleClass : MonoBehaviour {
    void Update() {
        transform.Translate(Vector3.forward * Time.deltaTime);
        transform.Translate(Vector3.up * Time.deltaTime, Space.World);
    }
}
```

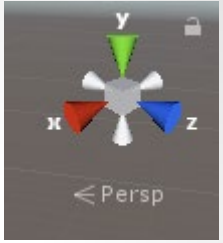
Forma abreviada de escribir el vector Vector3(0, 0, 1).

Tiempo en segundos que ha tardado en completar el último frame

Forma abreviada de escribir el vector Vector3(0, 1, 0).

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Transform.Rotate



- Método de la componente Transform.
- public void Rotate(Vector3 eulerAngles, Space relativeTo = Space.Self);
- Parámetros:
  - eulerAngles: rotación que se va a aplicar.
  - relativeTo: rotación local al objeto (Space.Self) o al Mundo (Space.World).

```
using UnityEngine;

public class ExampleClass : MonoBehaviour
{
    void Update()
    {
        // Rotate the object around its local X axis at 1 degree per second
        transform.Rotate(Vector3.right * Time.deltaTime);

        // ...also rotate around the World's Y axis
        transform.Rotate(Vector3.up * Time.deltaTime, Space.World);
    }
}
```

Forma abreviada de escribir Vector3(1, 0, 0).

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Ejemplo 1: Movimiento compuesto

- Se aplica al objeto un movimiento compuesto por rotación y traslación

```
public class Test : MonoBehavior {
        [SerializeField]
        private Vector3 rotationVelocity;

        void Start(){
                rotationVelocity = new Vector3(15, 0, 0);
        }

        void Update(){
                transform.Rotate(rotationVelocity * Time.deltaTime);
                transform.Translate(0.1f * Vector3.up * Time.deltaTime, Space.World);
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Ejemplo 2: Movimiento restringido

- Movimiento restringido entre una posición superior e inferior.

```
public class Test : MonoBehaviour {
        private bool goingUp;
        void Start(){
                goingUp=true;
        }
        void Update(){
                if (goingUp){
                        transform.Translate(Vector3.up * Time.deltaTime, Space.World);
                        if (transform.position.y >3){
                                goingUp = false;
                        }
                }
                else{
                        transform.Translate(Vector3.down * Time.deltaTime, Space.World);
                        if (transform.position.y <1){
                                goingUp = true;
                        }
                }
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Eventos asociados a una colisión

- En Unity, se produce una colisión cuando un GameObject con Collider entra en el espacio de otro GameObject con Collider (y al menos uno de los dos objetos tiene Rigidbody).
- Cuando hay una colisión, el motor de físicas llama a métodos con nombres específicos en cualquier script de los objetos implicados.
  - En el primer Update donde se detecta la colisión, se llama al método OnCollisionEnter.
  - Durante los Updates en los que se mantiene el contacto, se llama al método OnCollisionStay.
  - OnCollisionExit indica que el contacto se ha roto.

Ejemplo: Cambio de color de un obje que choca con otro

- Se cambia el color del material del objeto que choca con otro si se cumple una determinada condición (en este caso, si el tag del objeto es “Interactable”).
- Ejemplo: cubo que cae y choca con el suelo (el script es un componente del suelo)

```
public class Test : MonoBehavior {


        void OnCollisionEnter(Collision collision)
          {
              if (collision.gameObject.CompareTag("Interactable"))
              {
                 Renderer rend = collision.gameObject.GetComponent<Renderer>();
                 rend.material.color = Color.red;
              }
        }
}
```

Scripts asociados a colisiones (Trigger colliders)

- Cuando un collider con la opción “Is Trigger” marcada toca o se solapa con otro objeto:
  - No se genera la colisión.
  - Los objetos simplemente pasan uno a través del otro.
  - No se llama a los métodos OnCollision....
  - Se llama a los métodos análogos:
    - OnTriggerEnter
    - OnTriggerStay
    - OnTrigger Exit

Ejemplo: OnTriggerEnter

```
public class ExampleClass : MonoBehaviour {
    void OnTriggerEnter(Collider other) {
        Destroy(other.gameObject);
    }
}
```

- NOTA: En el ejemplo anterior, se usa el método Destroy:

  public static void Destroy(Object obj, float t = 0.0f);

  obj → GameObject o componente que se va a destruir

  t → parámetro opcional, tiempo de retardo antes de destruir el objeto.

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Modificaciones en GameObjects por medio de scripts

- Los scripts se pueden utilizar para hacer cambios en cualquier propiedad de los componentes de un GameObject:
  - La trasformación con transform.Translate y transform.Rotate.
  - El color del material del Renderer.
  - La masa de un Rigidbody.
  - …
- En general, cualquier cambio que se pueda hacer desde el editor se puede hacer desde un script.
- Ventaja: el script permite hacer los cambios de forma controlada:
  - Cambios graduales en el tiempo.
  - Cambios en respuesta a la entrada de usuario.

Acceder a componentes del propio GameObject

- En algunas ocasiones, un script necesita acceder a otros componentes del mismo GameObject.
- Cada componente es una instancia de una clase, por lo que se debe obtener una referencia a la instancia del componente con el que se quiere trabajar → Método **GetComponent**

```
void Start(){

        Rigidbody rb = GetComponent<Rigidbody>();

}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Modificar valores y acceder a métodos

- Una vez se tiene la referencia a la instancia del componente, ya es posible:
  a) Configurar valores de sus propiedades.
  b) Llamar a métodos proporcionados por el componente.

```
Rigidbody rb = GetComponent<Rigidbody>();
rb.mass = 10f; // (a) Cambia la masa del Rigidbody
rb.AddForce(Vector3.up*0.01f); // (b) Añade una fuerza al Rigidbody
```

Ejemplo: Añadir fuerzas

```
using UnityEngine;

public class ExampleClass : MonoBehaviour
{
    public float thrust;
    public Rigidbody rb;

    void Start()
    {
        rb = GetComponent<Rigidbody>();
    }

    void FixedUpdate()
    {
        rb.AddForce(0,0, thrust, ForceMode.Impulse);
    }
}
```

GetComponent: Devuelve la componente de tipo <Rigidbody> si el GameObject tiene este tipo de componente; si no, devuelve null.

Si no se indica este parámetro, se aplica el modo ForceMode.Force: se aplica fuerza continua sobre la masa en la dirección del vector de fuerza.
Con ForceMode.Impulse: se aplica una fuerza instantánea

Rigidbody.AddForce: Añade una fuerza al Rigidbody

Dirección del vector fuerza:

- 3 coordenadas: x,y,z
- Vector3(x,y,z)

Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Encontrar otros GameObjects

- En muchas ocasiones, querremos modificar propiedades de componentes de otros GameObjects.
- Hay distintos métodos para encontrar y acceder desde un script a otros GameObjects:
  - Método 1: Agregar una variable public GameObject al script.
  - Método 2: Buscar entre los GameObjects hijos el GameObject que se quiere modificar.
  - Método 3: Buscar GameObjects por nombre o etiqueta.

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Método 1: Agregar una variable public GameObject al script

```
public class Test : MonoBehavior {
        public GameObject otherObject;
        void Start(){
                otherObject.transform.position=new Vector3(-1f, 0f, -2f);
        }
}
```

- Agregar una variable public GameObject es la manera más directa de encontrar un GameObject.
- Esta variable estará visible en el Inspector como cualquier otra variable pública.
- De esta forma, es posible arrastrar el GameObject buscado desde la escena o desde la jerarquía para asignarlo.
- Una vez asignada, se puede acceder a sus componentes.

Método 1: Agregar una variable public GameObject al script

```
public class Test : MonoBehavior {
        public Transform otherTransform;
        void Start(){
                otherTransform.position=new Vector3(-1f, 0f, -2f);
        }
}
```

- También es posible agregar variables públicas de un tipo de componente.
- De esta forma, es posible acceder directamente a su componente en lugar de acceder al GameObject en sí mismo.

Comentarios generales sobre método 1

- El método 1 es el más cómodo si se trabaja con objetos individuales que van a tener conexiones permanentes con determinados objetos.
- Sin embargo, a veces es necesario acceder a un conjunto de objetos. Ejemplo: para acceder a objetos que representan todas las posibles posiciones iniciales de la cámara.
  - Esto es posible hacerlo con el método 1, pero es tedioso si hay que arrastrar todos los objetos correspondientes a todas las posibles posiciones iniciales.
  - Igualmente, para eliminar una posible posición inicial hay que quitar la variable que referencia al objeto que se elimina.
- Para casos como estos, y en general cuando se desee hacer conexiones con objetos en tiempo de ejecución desde los scripts, hay que utilizar los métodos 2 y 3.

Método 2: Encontrar objetos hijos

- El método consiste en hacer que los objetos a los que se quiere acceder sean hijos de un objeto padre.
- Se hace uso del **componente Transform** (todos los GameObjects tienen este componente).
- Ejemplo: Generar un vector de Transforms (Transform[]) y rellenarlo con las transformaciones de todos los GameObjects hijos.

```
public class Test : MonoBehavior {
        public Transform[] initialTransforms;
        void Awake(){
                initialTransforms = new Transform[transform.childCount];
                int i = 0;
                foreach (Transform t in transform){
                        Debug.Log(t.position);
                        initialTransforms[i++]=t;
                 }
        }
}
```

Método 2: Encontrar objetos hijos

- También haciendo uso del **componente transform** es posible encontrar un objeto hijo específico por el nombre utilizando el método transform.Find.
- Ejemplo: buscar un objeto hijo de nombre “Ball”

```
public class Test : MonoBehavior {
        [SerializeField]
        private GameObject ball;
        void Start(){
                Transform ballTransform = transform.Find(“Ball");
                ball=ballTransform.gameObject;
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Método 3: Encontrar objetos por nombre o etiqueta (Tag)

- En este caso, no es necesaria ninguna relación dentro de la jerarquía con el objeto buscado.
- Para recuperar objetos **por nombre**, se utiliza el método GameObject.Find.

```
public class Test : MonoBehavior {
        [SerializeField]
        private GameObject floor;
        void Start(){
                floor = GameObject.Find("Floor");
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Método 3: Encontrar objetos por nombre o etiqueta (Tag)

- Para recuperar objetos **por etiqueta**, se utilizan los métodos GameObject.FindWithTag y GameObject.FindGameObjectsWithTag.

```
public class Test : MonoBehavior {
        [SerializeField]
        private GameObject welcomeCharacter;
        [SerializeField]
        private GameObject[] characters;
        void Start(){
                welcomeCharacter = GameObject.FindWithTag("WelcomeCharacter");
                characters = GameObject.FindGameObjectsWithTag("Characters");
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Corrutina

- En general, los métodos se ejecutan completamente y devuelven el control al método que los llamó, con algunos valores de retorno.
  - Se ejecutan en una única actualización de frame.
- Si queremos que un método ejecute una secuencia de eventos a lo largo del tiempo, hay que usar corrutinas.
- Una corrutina es un método que puede:
  - Pausar su ejecución
  - Devolver el control a Unity.
  - Continuar en el siguiente frame desde donde pausó la ejecución.

Características de las corrutinas

- Si se utiliza una corrutina en un script, es necesario importer el espacio de nombres System.Collections
  - Incluir al principio del script “using System.Collections;”
- Su tipo de retorno es Inumerator
- Tiene un “yield return” incluido en su Código.
  - La línea donde está en “yield return” es el punto donde se pausa la ejecución y continua en el siguiente frame.
- Para empezar a ejecutar hay que llamar al método haciendo uso de StartCoroutine.
- Para detener una corrutina, hay dos métodos:
  - public void StopCoroutine(string methodName);
  - public void StopAllCoroutines()

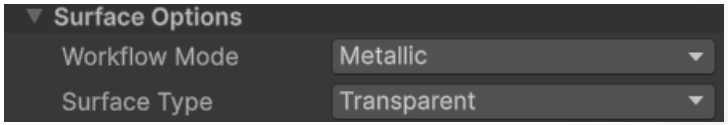
Ejemplo de Corrutina

```
using UnityEngine;
using System.Collections;
public class trigger : MonoBehaviour
{
    IEnumerator Fade(Renderer rend){
            Color c = rend.material.color;
            for (float alpha = 1f; alpha >= 0; alpha -= 0.1f)
            {
                c.a = alpha;
                rend.material.color = c;
                yield return null;
            }
}
…
```

Para lanzar la corrutina, incluir en algún método:

```
StartCoroutine(Fade(rend));
```

NOTA: Esta corrutina va bajando la transparencia del color del material. El material debe tener Surface Type= Transparent para poderlo visualizar.



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Retardos en la ejecución de la corrutina

- Uso de WaitForSeconds.

```
IEnumerator Fade(Renderer rend){
        Color c = rend.material.color;
        for (float alpha = 1f; alpha >= 0; alpha -= 0.1f)
        {
            c.a = alpha;
            rend.material.color = c;
            yield return new WaitForSeconds(3.0f);
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Ejemplo: Movimiento aleatorio de un objeto

```
using UnityEngine;
using System.Collections;
public class Test : MonoBehaviour {
        private Vector3 translation;
        bool movement=true;
        void Start(){
                translation = new Vector3(0, 0, 0);
                StartCoroutine(RandomMovement());
        }
        IEnumerator RandomMovement(){
                while (movement)
                {
                        translation.y=Random.Range(-0.1f, 0.1f);
                        transform.Translate(translation, Space.World);
                        yield return new WaitForSeconds(1.0f);
                }
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Instanciar prefabs en tiempo de ejecución

- Es posible crear instancias de un prefab en tiempo real durante la ejecución de la aplicación.
- El código necesitará una referencia al prefab:
  - Variable pública de tipo GameObject para almacenar la referencia al Prefab.
  - Asignar el prefab que se quiere utilizar en el Inspector.

```
public class InstantiatingPrefabs : MonoBehaviour {
        // Referencia al prefab. Hay que arrastrar un prefab a este campo en el Inspector.
        public GameObject myPrefab;

        void Start(){
                // Instanciar prefab en posición (0,0,0) y rotación (0,0,0)
                Instantiate(myPrefab, new Vector3(0,0,0), Quaternion.identity);
        }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Índice

0- Previo al tema: visualización de controladores en RV.

1- Creación de scripts.

2- Métodos de inicialización.

3- Métodos de actualización.

4- Primeros pasos: debug, consola.

5- Variables.

6- Modificaciones en las transformaciones.

7- Eventos asociados a detección de colisiones.

8- Acceso a componentes del propio GameObject.

9- Acceso a componentes de otros GameObjects.

10 - Uso de corrutinas.

11 - Creación de GameObjects en tiempo de ejecución a partir de prefabs.

12- Scripts para personalizar aspectos relacionados con las Meta SDK.

Acceso a la clase OVRCameraRig

- Para tener acceso a la clase desde un script, es necesario crear una variable de ese tipo y asociarle el [Building Block]Camera Rig desde el editor de Unity.

```
[SerializeField] OVRCameraRig cameraRig;
```

OVRCameraRig Class

Extends MonoBehaviour

This class is the core component of the OVRCameraRig prefab, which provides the transform object to represent the Oculus tracking space and the trackable objects.

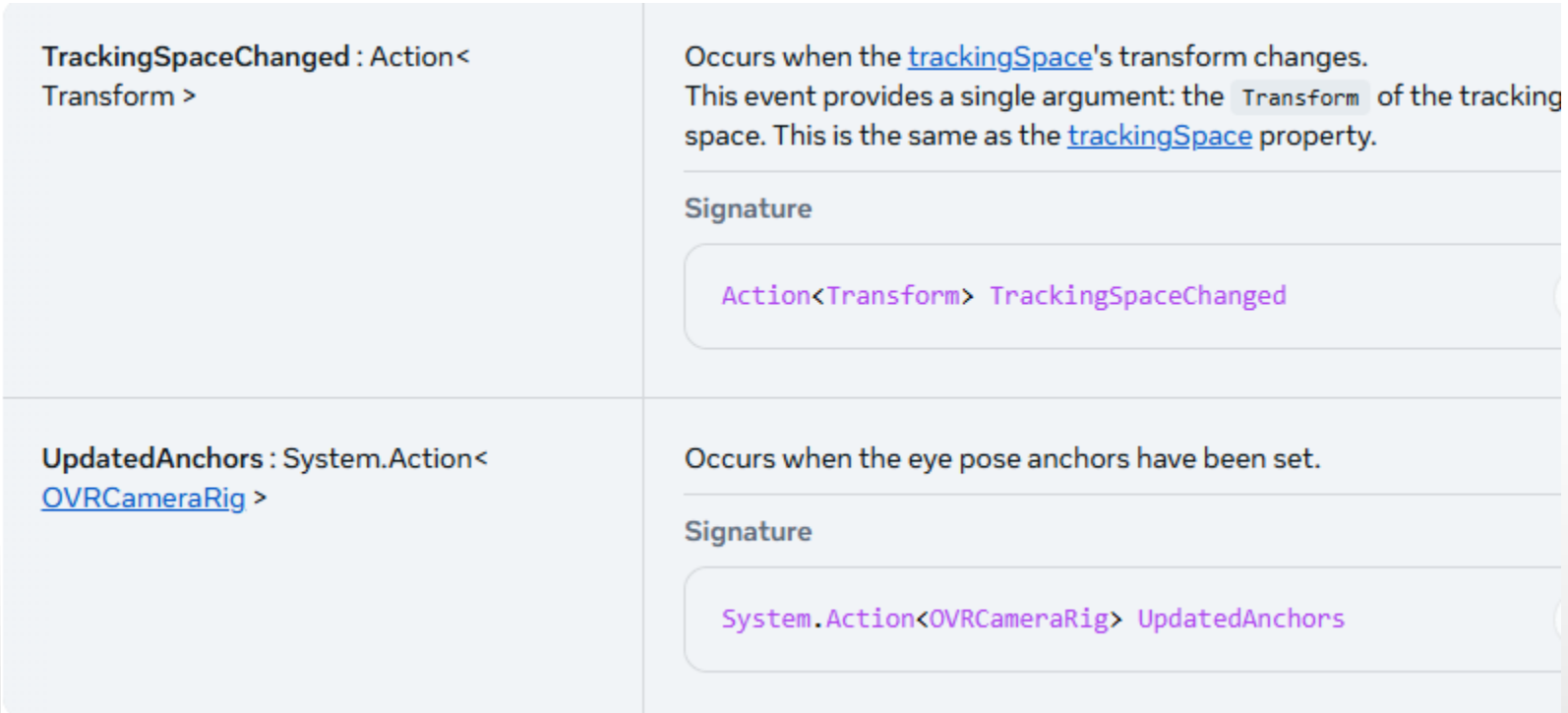
It contains a tracking space game object to fine-tune the relationship between the head tracking reference frame and your world. It also contains a center eye anchor, which is the main Unity camera, two anchor game objects for each eye, and left and right hand anchors for controllers.

https://developers.meta.com/horizon/reference/unity/v205/class_o_v_r_camera_rig/

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Eventos de la clase OVRCameraRig

- UpdatedAnchors: evento que se dispara cuando las anclas de ojos y manos ya se han actualizado a partir de los datos de tracking del frame actual.
- TrackingSpaceChanged: evento menos utilizado, se dispara cuando la transformación global de “trackingSpace” cambia, normalmente por un movimiento del objeto padre CameraRig (por ej., por teletransporte)



TrackingSpaceChanged : Action< Transform >

Occurs when the trackingSpace's transform changes.
This event provides a single argument: the `Transform` of the tracking space. This is the same as the trackingSpace property.

Signature

```
Action<Transform> TrackingSpaceChanged
```

UpdatedAnchors : System.Action< OVRCameraRig >

Occurs when the eye pose anchors have been set.

Signature

```
System.Action<OVRCameraRig> UpdatedAnchors
```

Suscripción a eventos

- Un evento es un mensaje que indica que ha ocurrido una acción (pulsación de botón, actualización de transformación, etc.).
- El suscriptor es el método que va a ser ejecutado cuando el evento ocurre.
- Por ejemplo: al mover la cabeza, se actualiza la posición de un panel
  - Evento: actualización de la transformación de la cabeza.
  - Suscriptor: método que actualiza la posición del panel.
- La sintaxis para suscribirse a un evento es la siguiente:

```
evento += método
```

- Si quiero suscribir un método (SeguirCabeza) al evento UpdatedAnchors, lo hago así:

```
cameraRig.UpdatedAnchors += SeguirCabeza;
```

- Este método no se llama manualmente en el código, se ejecuta solo cada vez que el evento se dispara.

Suscripción a eventos

- La suscripción a eventos se realiza en el método OnEnable(), cuando el objeto se activa.

```
void OnEnable(){
        cameraRig.UpdatedAnchors += SeguirCabeza;
}
```

- De forma abreviada, por ser un método con una sola instrucción, el método anterior se puede escribir así:

```
void OnEnable() => cameraRig.UpdatedAnchors += SeguirCabeza;
```

- MUY IMPORTANTE: es necesario eliminar la suscripción cuando el objeto se desactiva:

```
void OnDisable(){
        cameraRig.UpdatedAnchors -= SeguirCabeza;
}
```

- O de forma abreviada:

```
void OnDisable() => cameraRig.UpdatedAnchors -= SeguirCabeza;
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Ejemplo: Panel que sigue suavemente la mirada

- Al activarse el GameObject donde está el script, se vincula el método SeguirCabeza con el evento OVRCameraRig.UpdatedAnchors

```
using UnityEngine;

public class PanelDeSeguimiento : MonoBehaviour
{
   [SerializeField] OVRCameraRig cameraRig;
   [SerializeField] Transform panel;
   [SerializeField] float distancia = 1.2f;
   [SerializeField] float suavizado = 2f;

   // El Building Block "Camera Rig" dispara este evento cada
   // vez que actualiza los anchors de cabeza y manos
   void OnEnable() => cameraRig.UpdatedAnchors += SeguirCabeza;
   void OnDisable() => cameraRig.UpdatedAnchors -= SeguirCabeza;

   void SeguirCabeza(OVRCameraRig rig)
   {
      Transform cabeza = rig.centerEyeAnchor;
      Vector3 destino = cabeza.position + cabeza.forward * distancia;

      panel.position = Vector3.Lerp(panel.position, destino, Time.deltaTime * suavizado);
      panel.rotation = Quaternion.Slerp(panel.rotation, Quaternion.LookRotation(panel.position - cabeza.position),Time.deltaTime
* suavizado);
   }
}
```

Propiedad centerEyeAnchor del CameraRig

Lerp: Interpolación entre 2 vectores a y b según el factor t (valor entre 0 y 1)

Slerp: Interpolación entre 2 rotaciones a y b según el factor t (valor entre 0 y 1)

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

OVRInput

- Clase que proporciona un interfaz unificado para el acceso a datos de entrada de distintos tipos de controladores.

GetDown (virtualMask , controllerMask)

Gets the current down state of the given virtual button mask with the given controller mask.
Returns true if any masked button was pressed this frame on any masked controller and no masked button was previously down last frame.

Signature

```
static bool GetDown(Button virtualMask, Controller
controllerMask=Controller.Active)
```

Parameters

`virtualMask`: Button
`controllerMask`: Controller

Returns

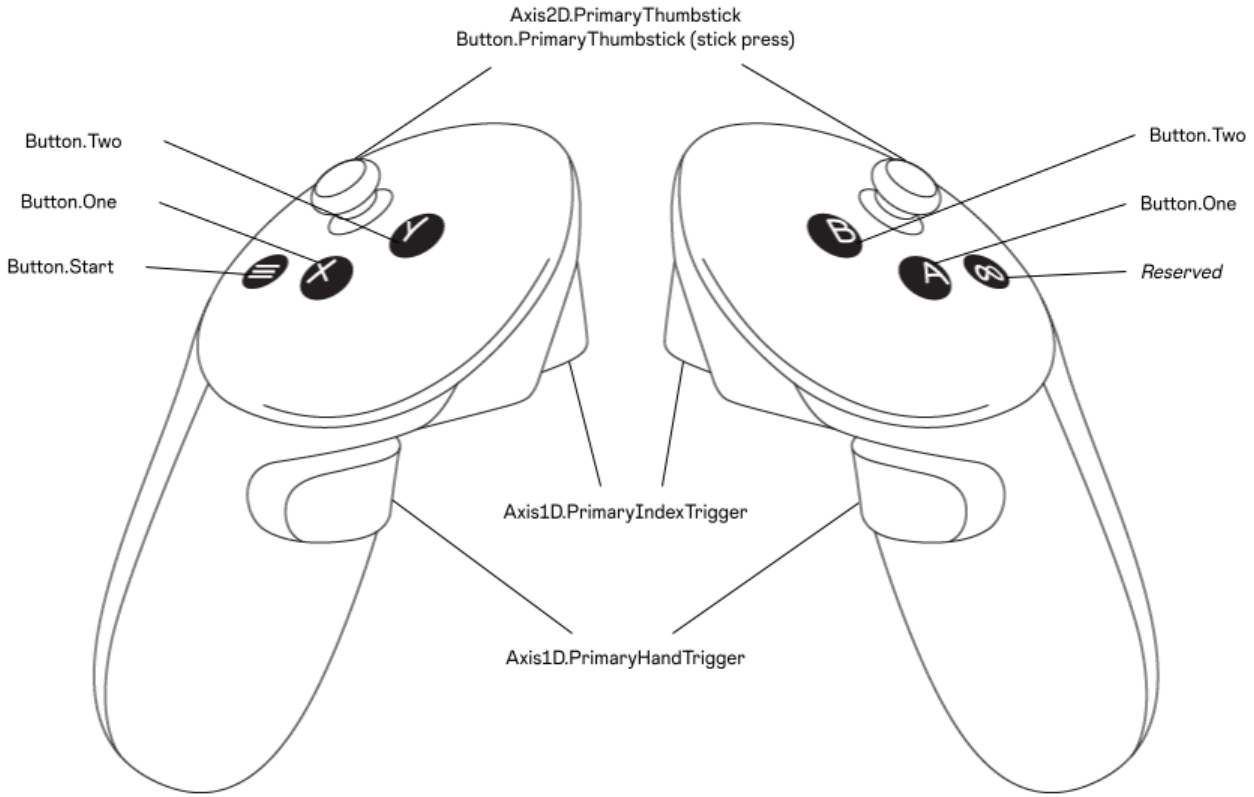
`bool`

Enum	LTouch (izquierdo)	RTouch (derecho)
`Button.One`	X	A
`Button.Two`	Y	B

EJEMPLO: pulsación del botón X del controlador izquierdo
OVRInput.GetDown(OVRInput.Button.One, OVRInput.Controller.LTouch)

https://developers.meta.com/horizon/reference/unity/v205/class_o_v_r_input

OVRInput



Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega

Ejemplo: objeto que desaparece al pulsar el botón X del controlador izquierdo

```
using UnityEngine;

public class DesaparecerConBoton : MonoBehaviour
{
   void Update()
   {
      if (OVRInput.GetDown(OVRInput.Button.One, OVRInput.Controller.LTouch))
      {
         gameObject.SetActive(false);
      }
   }
}
```

Departamento de Ingeniería Gráfica
Beatriz Rey Solaz
Alejandro Rodríguez Ortega